

바이브코딩 서비스 보안 점검 가이드

AI로 만든 학급 서비스 · 독서기록장 · 퀴즈앱 · 신청폼 · 대시보드 · 학생 포트폴리오 — 배포 전 자가 점검

이 가이드 사용법

이 문서는 **본인이 만든 서비스를 배포하기 전에**, **직접 한 번씩 따라 하며 점검하는 실무 가이드**입니다. 순서대로 읽으며 명령어를 복사·실행하면 됩니다.

1. 먼저 ‘**핵심 5가지**’ 표로 큰 그림을 잡습니다.
2. ‘**명령어로 직접 점검**’에서 터미널 명령어를 주소만 바꿔 실행합니다.
3. 터미널이 익숙하지 않으면 ‘**Claude Code · Codex로 점검 맡기기**’의 프롬프트를 그대로 붙여넣습니다.
4. ‘**배포 전 체크리스트**’에 빠진 항목이 없는지 확인합니다.
5. 문제를 발견하면 ‘**발견했을 때**’의 조치 순서를 따릅니다.

이 가이드는 **반드시 본인이 만든 서비스, 또는 운영자에게 명확히 허락받은 서비스에서만** 사용하세요. 남의 사이트를 점검하는 자료가 아닙니다.

바이브코딩은 서비스를 빠르게 만들어 주지만 **보안까지 책임져 주지는 않습니다**. 특히 요즘은 목표만 주면 알아서 움직이는 에이전트형 AI가 자동화된 도구로 인터넷을 쉬지 않고 훑기 때문에, ‘작아서 아무도 모를 것’이라는 가정은 더 이상 통하지 않습니다. 학교 서비스는 작은 구멍 하나가 곧바로 개인정보 사고로 이어집니다.

핵심 5가지

아래 다섯 가지에 모두 “**아니요**”라고 답할 수 있으면 기본 보안은 갖춘 것입니다. 하나라도 “**예**”라면 아직 학생에게 배포하면 안 됩니다.

점검 포인트	왜 위험한가	정상 / 위험
로그인 없이 데이터가 보이는가	주소만 알면 학생 데이터가 그대로 빠져나감	401·403·로그인 이동 / 200 + 데이터
주소 숫자만 바꿔 남의 데이터가 보이는가	/students/1 → /2 로 반 전체가 노출	본인 것만 보임 / 타인 기록 보임
비밀 키가 브라우저로 내려가는가	프론트 코드는 누구나 열어볼 수 있음	서버에만 있음 / JS·env 에 노출
인증번호·PIN을 무한히 시도할 수 있는가	시간만 있으면 결국 뚫림	횟수 제한 있음 / 제한 없음
학생 정보가 검색·공개 링크에 남는가	실명·학급이 검색엔진에 인덱싱됨	noindex·인증 / 공개 노출

명령어로 직접 점검 (주소만 바꿔 실행)

아래 명령어에서 **your-site.example** 부분만 본인 서비스 주소로 바꿔 터미널에 붙여넣습니다. 모두 **읽기 요청** 위주이며, 로그인 우회·비밀번호 추측·대량 요청은 하지 않습니다.

① HTTPS로만 접속되는가

```
curl -I http://your-site.example
```

정상: 301·302로 https 주소로 이동. HTTP로 로그인·대시보드가 열리면 위험.

왜 큰일나는가 공용 와이파이·학교망에서 로그인 정보가 그대로 노출될 수 있습니다.

② 보안 헤더가 있는가

```
curl -I https://your-site.example
```

Content-Security-Policy 가 없으면 우선 보완 대상. HSTS·X-Content-Type-Options 등도 함께 확인.

왜 큰일나는가 XSS가 생겼을 때 세션·입력값 탈취 위험이 커집니다.

③ 로그인 없이 관리자·대시보드가 열리는가

```
curl -i https://your-site.example/admin
```

```
curl -i https://your-site.example/dashboard
```

정상: 302·401·403. 위험: 200 + 관리자 화면·학생 목록.

왜 큰일나는가 로그인 화면이 따로 있어도, 서버가 막지 않으면 주소를 아는 사람은 그냥 들어옵니다.

④ API가 로그인 없이 데이터를 주는가

```
curl -i https://your-site.example/api/students
```

```
curl -i https://your-site.example/api/dashboard
```

정상: 401·403. 위험: 학생 목록·이메일·기록이 JSON으로 바로 반환.

왜 큰일나는가 화면에서 안 보여도 API가 열려 있으면 데이터는 이미 공개된 것과 같습니다.

⑤ 프런트 코드에 비밀값이 있는가

```
curl -s https://your-site.example/assets/app.js | rg -n  
"apiKey|secret|token|password|firebase|supabase|serviceRole|admin"
```

서비스 키키·관리자 키·DB 비밀번호는 **절대** 프런트에 있으면 안 됨. NEXT_PUBLIC_·VITE_ 접두사는 브라우저에 공개됨.

왜 큰일나는가 프런트 코드는 사용자 브라우저로 내려갑니다. 숨겼다고 생각해도 누구나 볼 수 있습니다.

⑥ .env 파일이 웹에서 열리는가

```
curl -I https://your-site.example/.env

curl -I https://your-site.example/.env.local
```

정상: 404•403. 위험: 200 + DB 주소•키 노출.

왜 큰일나는가 .env가 열리면 서비스 전체 열쇠를 공개한 것과 같습니다.

⑦ 개발용 문서가 열리는가 (Swagger•Actuator)

```
curl -I https://your-site.example/swagger-ui/index.html

curl -I https://your-site.example/actuator
```

운영 환경에서는 404•403 또는 관리자 인증 필요. 500이면 내부 오류가 밖으로 드러난 상태.

왜 큰일나는가 API 문서가 열리면 공격 도구가 ‘어디로 무엇을 요청할지’를 손쉽게 알게 됩니다.

Claude Code • Codex로 점검 말기기

터미널 명령어가 익숙하지 않다면, Claude Code나 Codex 같은 에이전트형 코딩 도구에게 점검을 시킬 수 있습니다. 아래 프롬프트를 **프로젝트 폴더 안에서** 그대로 붙여넣으면, 코드를 읽고 약한 곳을 찾아 설명해 줍니다.

먼저 알아 둘 것

에이전트는 ‘**목표를 주면 어떻게든 실행**’합니다. 점검을 시킬 때는 ‘찾아서 보고만 하고, 내 확인 없이 코드를 고치거나 배포•삭제하지 말 것’을 반드시 함께 지시하세요.

실제 학생 데이터가 들어 있는 운영 서버가 아니라, **로컬•테스트 환경에서 먼저** 돌리는 것이 안전합니다.

프롬프트 1 — 코드 전체 보안 점검

내 프로젝트 폴더에서 실행. 코드를 읽고 위험 요소를 표로 정리해 달라고 시키는 기본 점검입니다.

이 프로젝트는 교사가 학생용으로 배포할 웹 서비스야.

아래 항목을 코드에서 점검하고, 발견한 문제를 [위험도 / 파일·위치 / 설명 / 수정 방향] 표로 정리해줘.

중요: 너는 찾아서 보고만 해. 내 확인 없이 코드를 수정하거나 배포·삭제하지 마.

1. 로그인 없이 접근 가능한 페이지·API 가 있는지 (서버에서 권한 검사를 하는지)
2. 다른 사용자의 ID 로 바꿔 접근하면 남의 데이터가 보이는 구조인지 (IDOR)
3. API 키·서비스 를 키·DB 비밀번호가 프론트 코드나 깃 저장소에 들어가 있는지
4. 비밀번호를 평문으로 저장하는지, 인증번호·PIN·로그인 실패에 횟수 제한이 있는지
5. 학생 개인정보(이름·이메일·학급·생년월일)가 공개 페이지·로그·테스트 데이터에 남는지
6. 파일 업로드의 확장자·크기·실행 여부 검증이 있는지
7. 에러 화면에 stack trace·SQL·파일 경로가 노출되는지

프롬프트 2 — 비밀번호·키 유출만 빠르게

커밋·배포 전, 코드와 깃 기록에 키가 섞여 들어갔는지만 빠르게 확인할 때.

이 저장소의 코드와 git 커밋 기록(.env, 설정파일 포함)에서

API 키, 토큰, 비밀번호, 서비스 를 키, 인증 정보로 보이는 문자열을 찾아줘.

프론트엔드 번들에 노출되는 값(NEXT_PUBLIC_, VITE_, PUBLIC_ 접두사 포함)은 따로 표시해줘.

찾은 항목만 목록으로 보고하고, 파일을 수정하거나 키를 삭제하지는 마.

프롬프트 3 — 배포된 서비스 외부 점검

앞의 curl 점검을 에이전트가 대신 실행하게 하는 방식입니다. 본인 주소로 바꿔서 사용하세요.

https://your-site.example 는 내가 만든 서비스야.

아래를 curl 로 점검하고 결과(상태코드·헤더 위주)를 정리해줘.

읽기 요청만 하고, 로그인 우회·비밀번호 추측·대량 요청은 하지 마.

- http 로 접속 시 https 로 리다이렉트되는지
- 주요 보안 헤더(CSP, HSTS, X-Content-Type-Options)가 있는지
- /admin, /dashboard, /api/students 가 로그인 없이 200 으로 열리는지
- /.env, /.env.local 이 웹에서 열리는지
- /swagger-ui, /actuator 같은 개발용 경로가 노출되는지

각 항목을 [정상 / 확인 필요 / 위험] 으로 표시하고, 위험 항목은 이유를 설명해줘.

결과를 받은 뒤

에이전트가 ‘수정해줄까?’라고 물어도, **어디를 왜 고치는지 먼저 설명을 듣고** 직접 확인한 다음 적용하세요. 자동 수정이 멀쩡한 기능을 깨거나, 권한 로직을 잘못 바꾸는 경우가 있습니다.

배포 전 체크리스트

A. 로그인과 권한

- 로그인하지 않으면 대시보드가 열리지 않는다.
- 관리자 페이지는 관리자 계정만 열 수 있다.
- 학생 계정은 다른 학생 기록을 볼 수 없다.
- 교사 계정은 자기 반·자기 기관 데이터만 볼 수 있다.
- URL의 숫자 ID를 바꿔도 다른 사람 데이터가 보이지 않는다.
- 화면뿐 아니라 서버 API에서도 권한을 검사한다.

위험 신호

/students/1 을 /students/2 로 바꿨더니 다른 학생 정보가 보임

버튼은 숨겼지만 API를 직접 부르면 데이터가 나옴

관리자 메뉴만 숨기고 서버 권한 검사는 없음

B. 개인정보

- 이름·이메일·학년·반·생년월일을 꼭 필요한 만큼만 저장한다.
- 공개 페이지에 학생 실명·학급 정보가 나오지 않는다.
- 테스트 데이터에 실제 학생 정보를 넣지 않는다.

- 로그에 비밀번호·인증번호·토큰·개인정보가 남지 않는다.
- 삭제 요청이 들어오면 계정과 관련 데이터를 지울 수 있다.

C. 비밀번호와 인증

- 비밀번호를 서버에 평문으로 저장하지 않는다.
- 비밀번호 재설정 링크는 한 번만 쓰이고 짧은 시간 뒤 만료된다.
- 인증번호는 짧은 시간 뒤 만료된다.
- 로그인·PIN·인증번호 실패에 횟수 제한이 있다.
- 세션 쿠키에 HttpOnly · Secure · SameSite 가 붙어 있다.

확인: `curl -i https://your-site.example/login` 의 Set-Cookie 줄에서 HttpOnly·Secure·SameSite 를 확인합니다.

D. API와 데이터베이스

- API는 로그인 여부만 보지 않고 역할·소유권까지 확인한다.
- `userId·studentId·classId` 등 클라이언트가 보낸 값을 그대로 믿지 않는다.
- DB 조회 조건에 현재 로그인 사용자·기관 조건이 들어간다.
- 관리자 API는 일반 사용자 토큰으로 호출할 수 없다.
- 공개 API에는 개인정보가 없다.

E. 파일 업로드

- 업로드는 확장자와 MIME 타입을 모두 확인한다.
- 업로드 파일 크기 제한이 있다.
- 업로드된 파일을 서버 코드처럼 실행하지 않는다.
- 파일명에 개인정보가 들어가지 않는다.
- 공개 URL과 비공개 URL을 구분한다.

F. 외부 서비스 키

- OpenAI·Google·Firebase·Supabase·S3 키가 프론트 코드에 없다.
- 서비스 토큰 키·관리자 키는 서버에서만 쓴다.
- 키가 유출되면 바로 폐기하고 새로 발급한다.
- 무료 요금제라도 사용량 제한을 둔다.

G. CORS와 에러 화면

- Access-Control-Allow-Origin 에 * 와 `credentials:true` 조합을 쓰지 않는다.
- 실제 서비스 도메인만 허용하고, 관리자 API는 더 엄격히 제한한다.
- 에러 화면에 DB 오류·파일 경로·stack trace·SQL 문장이 보이지 않는다.
- 사용자에게는 짧은 오류 메시지만, 자세한 오류는 서버 로그에만 남긴다.

선생님이 특히 조심할 기능

학급 초대 링크

- 링크만 알면 누구나 들어오는 구조는 위험합니다.
- 초대 링크에는 만료 시간을 둡니다.
- 초대 후 교사 승인 또는 코드 확인 절차를 둡니다.

학생 결과 공유 링크

- 공개 링크에는 실명·학급·학교명을 빼는 편이 안전합니다.
- 꼭 필요하면 비밀번호나 만료 시간을 둡니다.

AI 피드백 기능

- 학생 입력이 외부 AI 서비스로 전송되는지 확인합니다.
- 민감한 개인정보를 AI 프롬프트에 넣지 않습니다.
- 보호자·학교 정책상 안내가 필요한지 확인합니다.

관리자 대시보드

- 관리자 주소를 ‘숨기는 것’은 보안이 아닙니다.
- 서버에서 관리자 권한을 반드시 확인합니다.
- 관리자 API에는 별도의 권한 검사를 둡니다.

문제를 발견했을 때

심각 — 즉시 조치

관리자·학생 데이터 API가 로그인 없이 열림 · .env·관리자 키 공개 · URL 변경만으로 타인 데이터 노출

1. 서비스를 잠시 내리거나 해당 경로를 막습니다.
2. 노출된 키·토큰을 폐기하고 새로 발급합니다.
3. 데이터 접근 로그를 확인하고, 실제 학생 정보 노출 여부를 점검합니다.
4. 필요하면 학교·기관 개인정보 담당자와 상의합니다.

가벼움 — 다음 배포에 반영

CSP 없음 · robots.txt 민감 경로 노출 · 로그인 페이지 검색 노출 · Swagger 500 반환

→ 보안 헤더·라우팅 정책 반영, 오류 응답을 404·403으로 정리, 인증 페이지에 noindex,nofollow 적용.

마지막 점검 — 모두 “아니요”여야 합니다

- 로그인하지 않아도 학생 데이터가 보이나요?
- 주소의 숫자만 바뀌도 다른 사람 데이터가 보이나요?
- 프론트 코드에 관리자 키나 DB 키가 있나요?
- 인증번호·PIN·로그인 실패를 계속 시도할 수 있나요?
- 오류 화면이 서버 내부 정보를 보여주나요?
- 검색엔진이 로그인·회원가입·비밀번호 찾기 페이지를 가져가나요?

하나라도 “예” 라면, 아직 학생에게 배포하면 안 됩니다.